Assignment No-1 Multiplication of Two Matrix

In [2]:
```
a=[[4,3,5],[9,8,7]]
b=[[1,2,3],[4,5,6]]
Result=[[0 for i in range(len(b[0]))] for j in range(len(a))]
for i in range(len(a)):
  for j in range(len(b[0])):
    for k in range(len(b)):
      Result[i][j]+=a[i][k]*b[k][j]
print("Multiplication of Two Matrix is : ",Result)
```

```
Multiplication of Two Matrix is :  [[16, 23, 30], [41, 58, 75]]
```

Explination : in this code I use 3 for loop for multipling all element of two matrix and store it in 3rd result matrix.

1 Scaler Multiplication

In [3]:
```
mat=[[1,2,3],[4,5,6]]
n=int(input("Enter the Number to Multiplay : "))
for i in range(len(mat)):
  for j in range(len(mat[0])):
    mat[i][j]=n*mat[i][j]
print("Matrix After Scaler Multiplication : ",mat)
```

```
Matrix After Scaler Multiplication :  [[5, 10, 15], [20, 25, 30]]
```

Explination - in this code I simple multiply all element in matrix by scaler element

Subtraction of Two Matrix :

In [3]:
```
a=[[4,3,5],[9,8,7]]
b=[[1,2,3],[4,5,6]]
Result=[]
if len(a)==len(b) and len(a[0])==len(b[0]):
  for i in  range(len(a)):
    row=[]
    for j in range(len(a[0])):
      row.append(a[i][j]-b[i][j])
    Result.append(row)
print("Subtraction of Two Matrix is : ",Result)
```

```
Subtraction of Two Matrix is :  [[3, 1, 2], [5, 3, 1]]
```

Explination : This code subtracts corresponding elements of two matrices a and b stores the result in a new matrix.

Addition of two matrix

In [4]:
```
a=[[1,2,3],[4,5,6]]
b=[[1,2,3],[4,5,6]]
Result=[]
if len(a)==len(b) and len(a[0])==len(b[0]):
  for i in  range(len(a)):
    row=[]
    for j in range(len(a[0])):
      row.append(a[i][j]+b[i][j])
    Result.append(row)
print("Addition of Two Matrix is : ",Result)
```

```
Addition of Two Matrix is :  [[2, 4, 6], [8, 10, 12]]
```

Explination : Is also as subtraction only change is sign inside the loop

ASSIGNMENT NO -2 Eigenvalues and Eigenvectors Calculation

In [4]:
```
A = [[2, 0], [1, 3]]

a = A[0][0]
b = A[0][1]
c = A[1][0]
d = A[1][1]

a1 = 1
b1 = -(a + d)
c1 = a * d - b * c

x1 = (-b1 + ((b1**2) - 4 * a1 * c1)**0.5) / 2
x2 = (-b1 - ((b1**2) - 4 * a1 * c1)**0.5) / 2

print("Eigenvalue 1 =", x1)
print("Eigenvalue 2 =", x2)

def eigenvector(matrix, eigenvalue):
    a = matrix[0][0] - eigenvalue
    b = matrix[0][1]
    c = matrix[1][0]
    d = matrix[1][1] - eigenvalue
    if b != 0:
        return [1, -a / b]
    elif c != 0:
        return [-d / c, 1]
    else:
        return [1, 0]

v1 = eigenvector(A, x1)
v2 = eigenvector(A, x2)

print("Eigenvector for X1:", v1)
print("Eigenvector for X2:", v2)
```

```
Eigenvalue 1 = 3.0
Eigenvalue 2 = 2.0
Eigenvector for X1: [-0.0, 1]
Eigenvector for X2: [-1.0, 1]
```

Explination: This code find eigenvalues of 2×2 matrix using quadratic formula and then calculate eigenvectors for each eigenvalue. It first take matrix elements, form equation,

solve roots, then check condition and display vector related to eigenvalue.

ASSIGNMENT NO -3 Gaussian elimination method

In [6]:
```
a=[[1,5,2],[4,2,8],[2,3,1]]
b=[6,5,4]
n=len(b)
for i in range(n):
    max_row = i
    for k in range(i + 1, n):
        if abs(a[k][i]) > abs(a[max_row][i]):
            max_row = k
    a[i], a[max_row] = a[max_row], a[i]
    b[i], b[max_row] = b[max_row], b[i]

    pivot = a[i][i]

    if pivot == 0:
        print("Matrix is singular, cannot perform Gaussian elimination.")
        break

    for j in range(n):
        a[i][j] = a[i][j] / pivot
    b[i] = b[i] / pivot

    for k in range(n):
        if(i != k):
            factor = a[k][i]
            for j in range(n):
                a[k][j] = a[k][j] - factor * a[i][j]

print("Matrix a after Gaussian elimination:")
for row in a:
    print(row)
print("Vector b after Gaussian elimination:")
print(b)
```

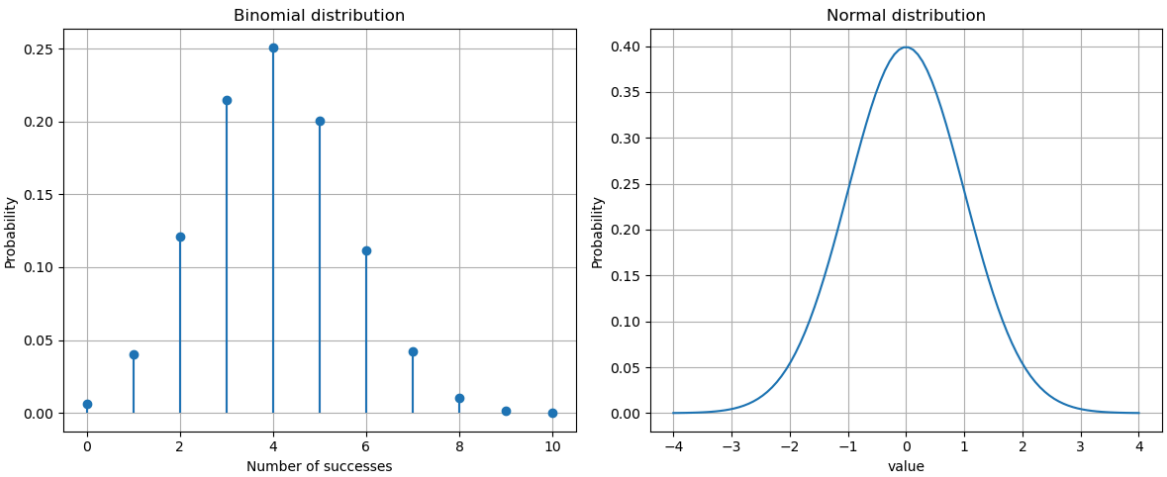
```
Matrix a after Gaussian elimination:
[1.0, 0.0, 0.0]
[0.0, 1.0, 0.0]
[-0.0, -0.0, 1.0]
Vector b after Gaussian elimination:
[1.25, 1.3333333333333333, -1.3333333333333333]
```

Explination : This code Gaussian elimination method use karke equation solve karta hai. Pehle row swap karke pivot set karta hai, phir column ke baki element zero banata hai, aur last me vector b me solution milta hai.

ASSIGNENT NO -4 Binomial distribution and Normal distribution

In [7]:
```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

n=10
p=0.3+.1

x_bino=np.arange(0,n+1)
```

```
pmf_bino=stats.binom.pmf(x_bino,n,p)

mu=0
sigma=1

x_norm=np.linspace(-4,4,100)
pmf_norm=stats.norm.pdf(x_norm,mu,sigma)

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.stem(x_bino,pmf_bino,basefmt=" ")
plt.title("Binomial distribution")
plt.xlabel("Number of successes")
plt.ylabel("Probability")
plt.grid(True)

plt.subplot(1,2,2)
plt.plot(x_norm,pmf_norm)
plt.title("Normal distribution")
plt.xlabel("value")
plt.ylabel("Probability")
plt.grid(True)


plt.tight_layout()
plt.show()
```

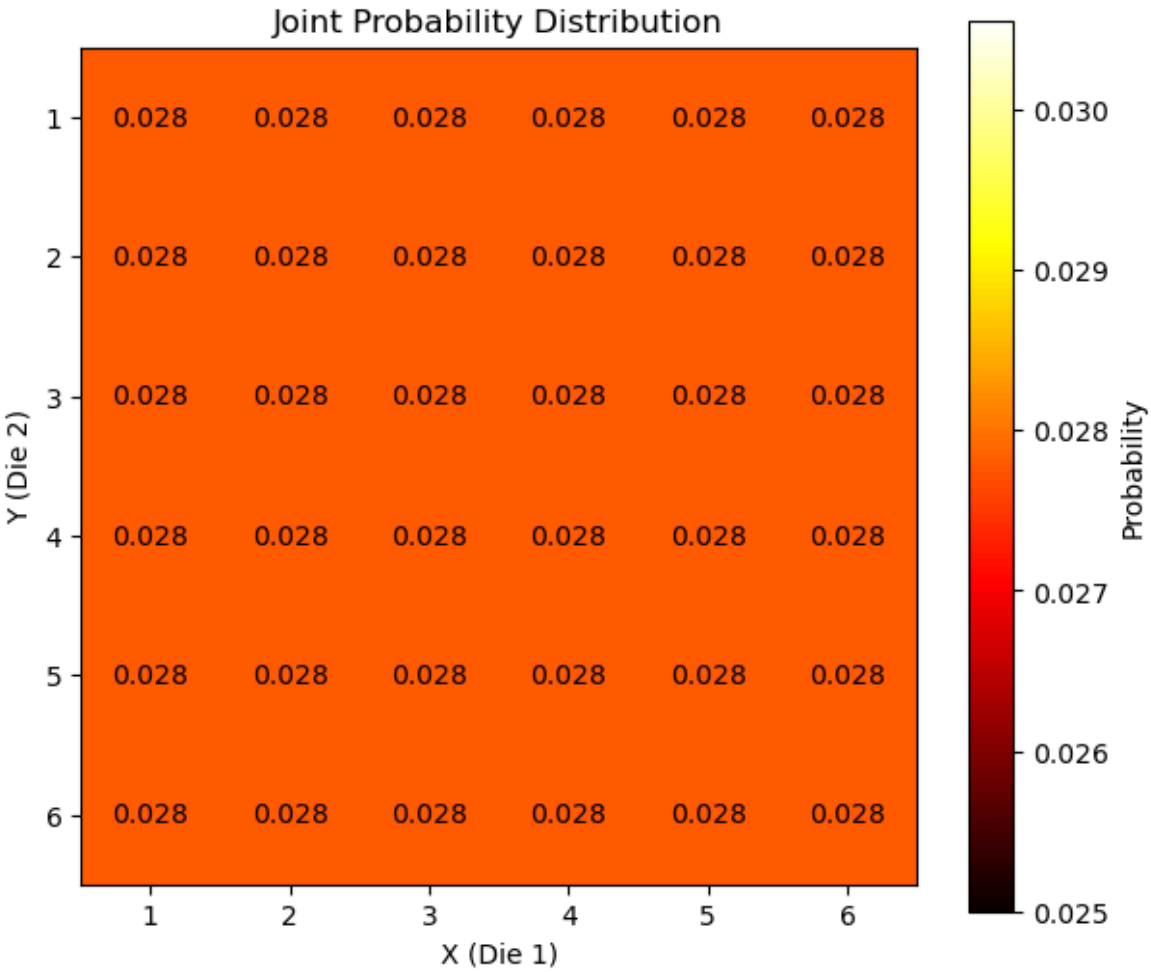


Explination : This code draw two graphs: one for Binomial distribution (n=10, p=0.4) and one for Normal distribution (mean=0, sigma=1). It calculate probabilities and show them side by side using matplotlib.

ASSIGNMENT NO- 5 Joint probability distribution

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt

die_outcome=[1,2,3,4,5,6]

sample_space=np.array([(x,y) for x in die_outcome for y in die_outcome])

joint_pmf={}
for (x,y) in sample_space:
  joint_pmf[(x,y)]=1/36
```

```
pmf_matrix=np.zeros((6,6))

for x in die_outcome:
  for y in die_outcome:
    pmf_matrix[x-1,y-1]=joint_pmf[(x,y)]

plt.figure(figsize=(7,6))
plt.imshow(pmf_matrix,cmap='hot',interpolation='nearest')

for i in range(6):
  for j in range(6):
    plt.text(j,i,'0.028',ha='center',va='center',color='black')

plt.title('Joint Probability Distribution')
plt.xlabel('X (Die 1)')
plt.ylabel('Y (Die 2)')
plt.colorbar(label='Probability')
plt.xticks(np.arange(6),die_outcome)
plt.yticks(np.arange(6),die_outcome)
plt.show()
```

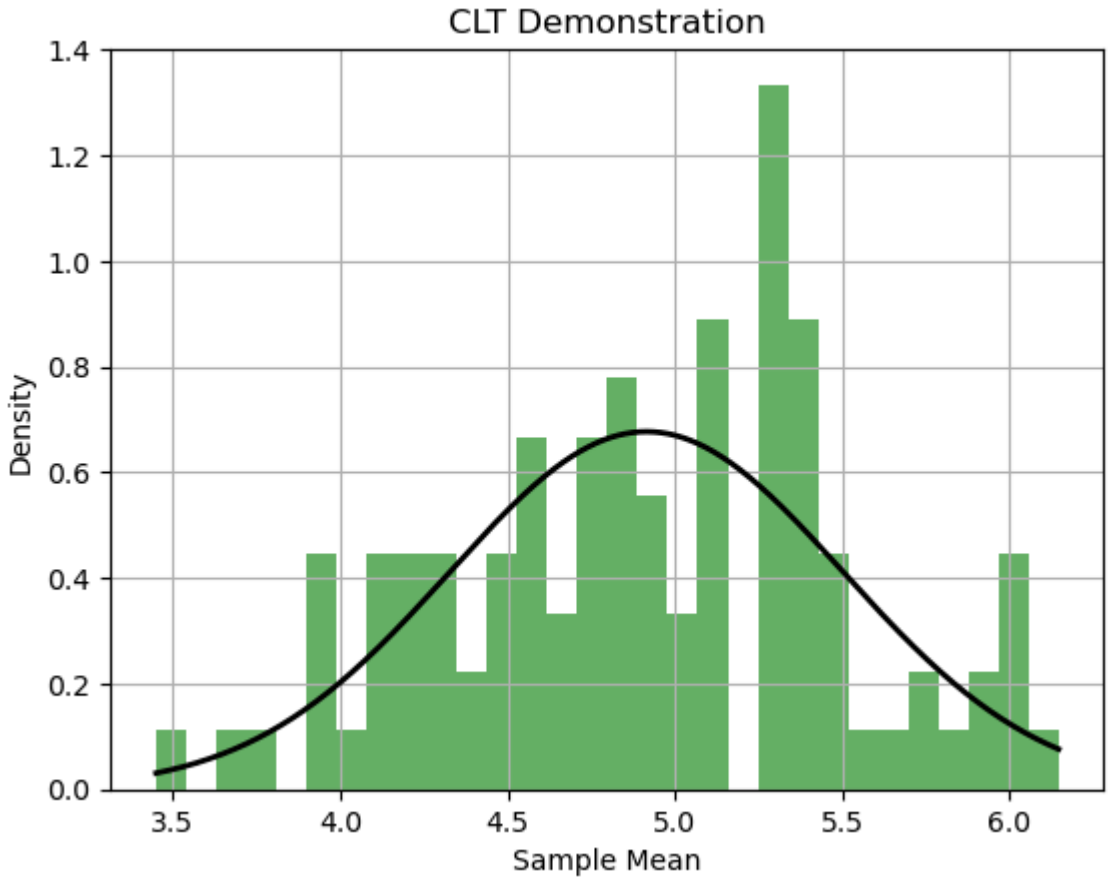


Explination : This code calculate and show joint probability distribution for rolling two dice. It first create all 36 possible outcomes, assign equal probability 1/36 to each pair, store in a 6×6 matrix, and then plot a heatmap with numbers on each cell.This helps visualize how likely each combination of dice rolls is.

ASSIGNMENT NO - 6

In [5]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

population = np.random.uniform(0, 10, 100000)

n = 30
num_samples = 100

sample_means = []
for i in range(num_samples):
    sample = np.random.choice(population, n)
    sample_means.append(np.mean(sample))

sample_means = np.array(sample_means)

mean_sample = np.mean(sample_means)

print("Mean of sample means: ", mean_sample)

std_pop = np.std(population)
std_sample = np.std(sample_means) #std_pop/n**0.5

print("Standard deviation of population: ", std_pop)
print("Standard deviation of sample means: ", std_sample)

plt.hist(sample_means, bins=30, density=True, alpha=0.6, color='g')

x = np.linspace(min(sample_means),max(sample_means), 1000)
pdf = norm.pdf(x, mean_sample, std_sample)
plt.plot(x, pdf, 'k', linewidth=2)
plt.title("CLT Demonstration")
plt.xlabel("Sample Mean")
plt.ylabel("Density")
plt.grid(True)
plt.show()
```

```
Mean of sample means:  4.914974644249048
Standard deviation of population:  2.894053962826204
Standard deviation of sample means:  0.5889938168687109
```

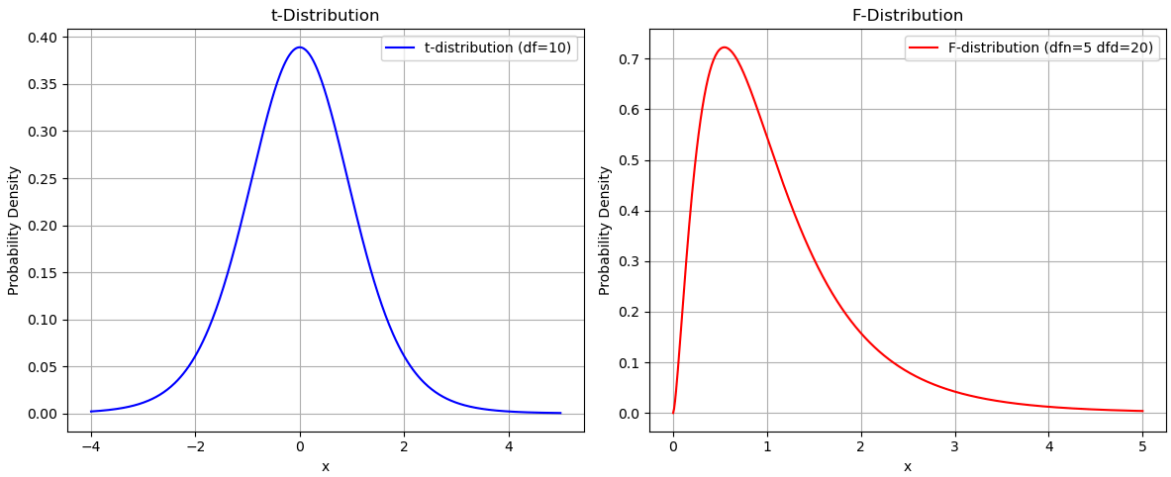


Explination : This code make many random samples from 0 to 1 and find mean and standard deviation for each sample. Then it use smooth curve to show how these means and std devs are spread. It help to see overall pattern of sample statistics and how they behave for big number of samples.

ASSIGNMENT NO - 7 t-DISTRIBUTION, f-DISTRIBUTION

In [10]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import t, f


# t-DISTRIBUTION

# Degrees of freedom (parameter for t-distribution)
df_t = 10

# Generate x values (range of values for plotting the curve)
x_t = np.linspace(-4, 5, 500)

# Probability Density Function (PDF) for t-distribution
pdf_t = t.pdf(x_t, df_t)

# F-DISTRIBUTION

# Degrees of freedom (two parameters for F-distribution)
dfn, dfd = 5, 20 # numerator and denominator degrees of freedom

# Generate x values (range of values for plotting the curve)
x_f = np.linspace(0, 5, 500)
```

```
# Probability Density Function (PDF) for F-distribution
pdf_f = f.pdf(x_f, dfn, dfd)

# PLOTTING BOTH DISTRIBUTIONS

plt.figure(figsize=(12, 5))

# Plot t-distribution
plt.subplot(1, 2, 1)
plt.plot(x_t, pdf_t, 'b', label="t-distribution (df=10)")
plt.title("t-Distribution")
plt.xlabel("x")
plt.ylabel("Probability Density")
plt.legend()
plt.grid(True)

#Plot F-distribution
plt.subplot(1, 2, 2)
plt.plot(x_f, pdf_f, 'r', label="F-distribution (dfn=5 dfd=20)")
plt.title("F-Distribution")
plt.xlabel("x")
plt.ylabel("Probability Density")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Explination: This code plot t-distribution and F-distribution side by side. It first calculate their probability densities for given degrees of freedom,then draw smooth curves so we can see how values are likely to occur in each distribution.

Assignment No-8 Z-test

In [6]:
```
import numpy as np
from scipy.stats import norm
def z_test(sample, mu0, alpha, sigma=None, alternative='two-sided'):
    sample = np.array(sample)
    n = len(sample)
    x_bar = np.mean(sample)
    if sigma is None:
        sigma = np.std(sample, ddof=1) # Sample standard deviatio
    z_stat = (x_bar - mu0) / (sigma / np.sqrt(n))
    # Calculate p-value based on the alternative hypothesis
```

```
    if alternative == "two-sided":
        p_value = 2 * (1 - norm.cdf(abs(z_stat)))
    elif alternative == "greater":
        p_value = 1 - norm.cdf(z_stat)
    elif alternative == "less":
        p_value = norm.cdf(z_stat)
    else:
        raise ValueError("Alternative must be 'two-sided', 'greater', or 'less'.
    reject = p_value <= alpha
    return {
        "Sample mean": x_bar,
        "Hypothesized mean": mu0,
        "z-statistic": z_stat,
        "p-value": p_value,
        "Conclusion": "Reject H0" if reject else "Fail to reject H0"
    }

data = [52, 50, 53, 49, 48, 51, 54, 50, 52, 49]
mu0 = 50
alpha = 0.05
result = z_test(data, mu0, alpha, alternative='two-sided')
for key, value in result.items():
    print(f"{key}: {value}")

# This code performs a Z-test to check if the sample mean is significantly diffe
# population mean . It calculates the Z-statistic using the sample mean, standar
# then finds the p-value based on the test type.
```

```
Sample mean: 50.8
Hypothesized mean: 50
z-statistic: 1.3093073414159497
p-value: 0.19043026382552575
Conclusion: Fail to reject H0
```

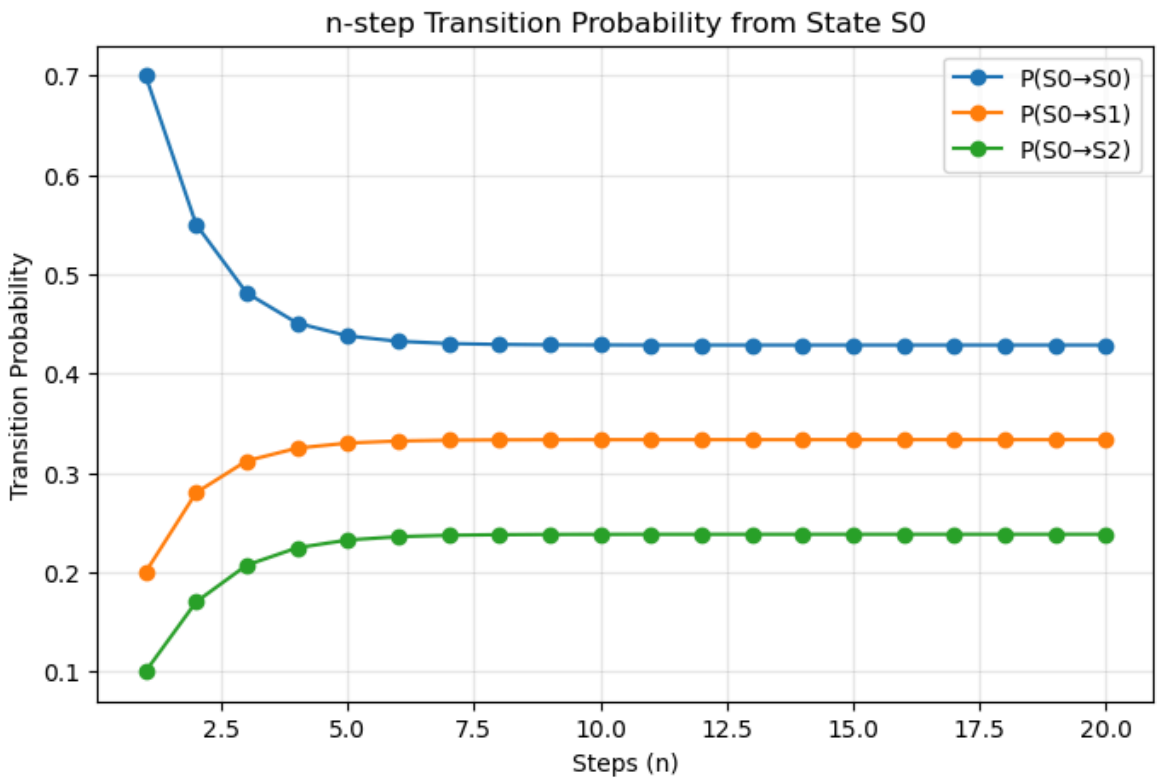
Assignment No- 9 Markov chain

In [5]:

```
import numpy as np
import matplotlib.pyplot as plt
def validate_transition_matrix(P):
    P = np.array(P)
    if P.shape[0] != P.shape[1] or P.ndim != 2:
        raise ValueError("Transition matrix must be square.")
    if not np.allclose(P.sum(axis=1), 1):
        raise ValueError("Each row of transition matrix must sum to 1.")
    if np.any(P < 0):
        raise ValueError("Transition matrix cannot have negative probabilities."
    return P
def n_step_transition_probability(P, n):
    P = validate_transition_matrix(P)
    return np.linalg.matrix_power(P, n)
# Example usage
if __name__ == "__main__":
    # Define transition matrix for 3-state Markov chain
    P = np.array([
        [0.7, 0.2, 0.1],
        [0.1, 0.6, 0.3],
        [0.4, 0.2, 0.4]
    ])
    steps = [1, 2, 5, 10, 20]
    results = {}
```

```
    for n in steps:
        Pn = n_step_transition_probability(P, n)
        results[n] = Pn
        print(f"\n{n}-step transition probability matrix:\n{Pn}")
    # Visualization: show how probability of going from S0 evolves
    state_names = ["S0", "S1", "S2"]
    max_n = 20
    evolution = [n_step_transition_probability(P, n)[0] for n in range(1, max_n+
    evolution = np.array(evolution)
    print("\n evolution:\n",evolution)
    plt.figure(figsize=(8, 5))
    for i in range(P.shape[0]):
        plt.plot(range(1, max_n+1), evolution[:, i], marker="o",
                 label=f"P(S0→{state_names[i]})")
    plt.xlabel("Steps (n)")
    plt.ylabel("Transition Probability")
    plt.title("n-step Transition Probability from State S0")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()

# This code works with a Markov chain and shows how the transition probabilities
# steps. It first checks if the given transition matrix is valid. Then it calcul
# transition matrix by raising the matrix to the power of n.
```

```
1-step transition probability matrix:
[[0.7 0.2 0.1]
 [0.1 0.6 0.3]
 [0.4 0.2 0.4]]

2-step transition probability matrix:
[[0.55 0.28 0.17]
 [0.25 0.44 0.31]
 [0.46 0.28 0.26]]

5-step transition probability matrix:
[[0.43777 0.32992 0.23231]
 [0.41191 0.34016 0.24793]
 [0.43534 0.32992 0.23474]]

10-step transition probability matrix:
[[0.42867376 0.33329838 0.23802786]
 [0.42837099 0.33340324 0.23822577]
 [0.42866785 0.33329838 0.23803377]]

20-step transition probability matrix:
[[0.42857144 0.33333333 0.23809523]
 [0.42857141 0.33333334 0.23809525]
 [0.42857144 0.33333333 0.23809523]]

 evolution:
 [[0.7        0.2        0.1       ]
 [0.55       0.28       0.17      ]
 [0.481      0.312      0.207     ]
 [0.4507     0.3248     0.2245    ]
 [0.43777    0.32992    0.23231   ]
 [0.432355   0.331968   0.235677  ]
 [0.4301161  0.3327872  0.2370967 ]
 [0.42919867 0.33311488 0.23768645]
 [0.42882514 0.33324595 0.23792891]
 [0.42867376 0.33329838 0.23802786]
 [0.42861261 0.33331935 0.23806804]
 [0.42858798 0.33332774 0.23808428]
 [0.42857807 0.3333311  0.23809083]
 [0.42857409 0.33333244 0.23809347]
 [0.4285725  0.33333298 0.23809453]
 [0.42857186 0.33333319 0.23809495]
 [0.4285716  0.33333328 0.23809512]
 [0.4285715  0.33333331 0.23809519]
 [0.42857146 0.33333332 0.23809522]
 [0.42857144 0.33333333 0.23809523]]
```



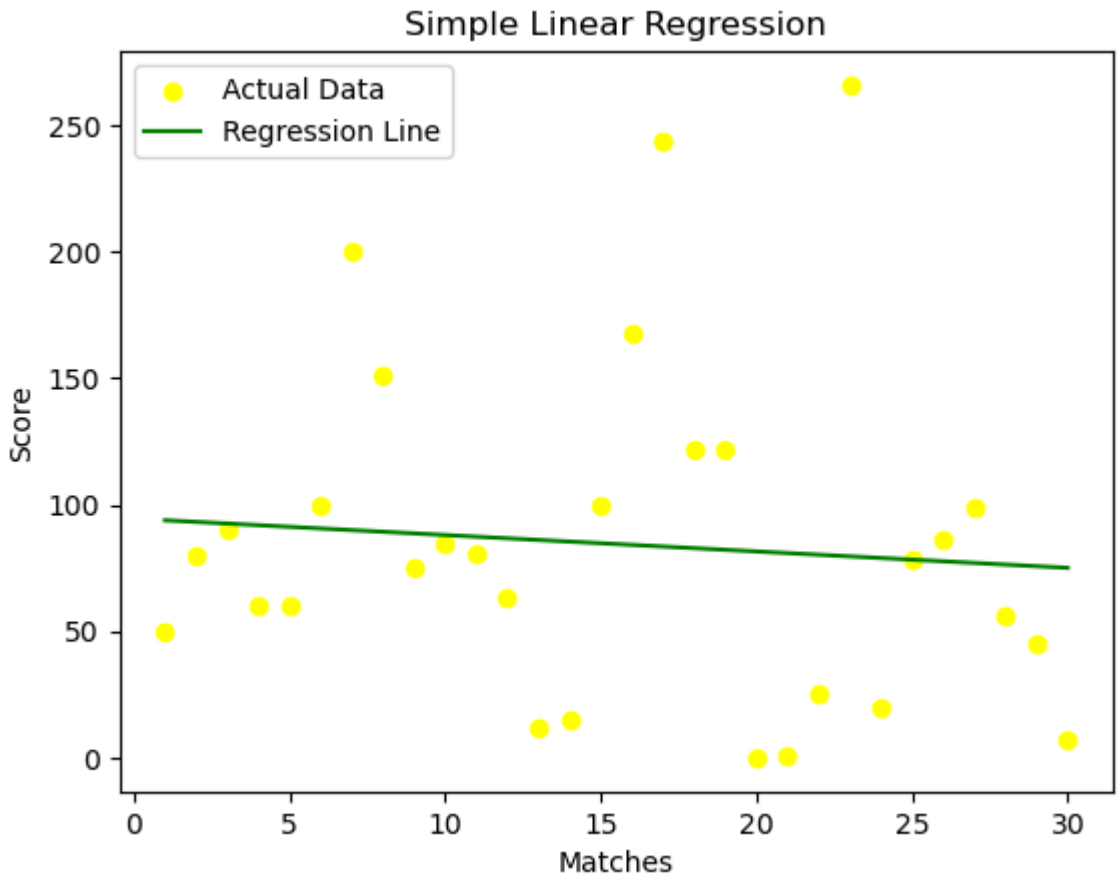
ASSIGNMENT NO 3 Unit 5 & 6

ASSIGNMENT NO - 10 Simple Linear Regression

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

data = pd.DataFrame({
    "Matches": [1,2,3,4,5,6,7,8,9,10,
                11,12,13,14,15,16,17,18,19,20,
                21,22,23,24,25,26,27,28,29,30],
    "Score": [50,80,90,60,60,100,200,151,75,85,
              81,63,12,15,100,168,244,122,122,0,1,
              25,266,20,78,86,99,56,45,7]
})

data.to_csv("salary_dataset.csv", index=False)

X = data["Matches"].values.reshape(-1, 1)  # Fixed typo here
y = data["Score"]

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_s

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print(f"Regression Equation: Score = {model.coef_[0]:.2f} * Matches + {model.int
print("Mean Squared Error (MSE): ", mse)
print("R Score: ", r2)
print()

plt.scatter(X, y, color="yellow", label="Actual Data")
plt.plot(np.sort(X, axis=0), model.predict(np.sort(X, axis=0)), color="green", l
plt.xlabel("Matches")
plt.ylabel("Score")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
```

```
Regression Equation: Score = -0.65 * Matches + 94.53
Mean Squared Error (MSE):  2108.5483493731763
R Score:  0.05455683816467782
```



This code applies Simple Linear Regression to find the relationship between matches played and scores. It trains a model to predict scores, checks its accuracy using MSE and R^2 values, and prints the regression equation. Finally, it plots the actual data points and the best-fit regression line for clear visualization.

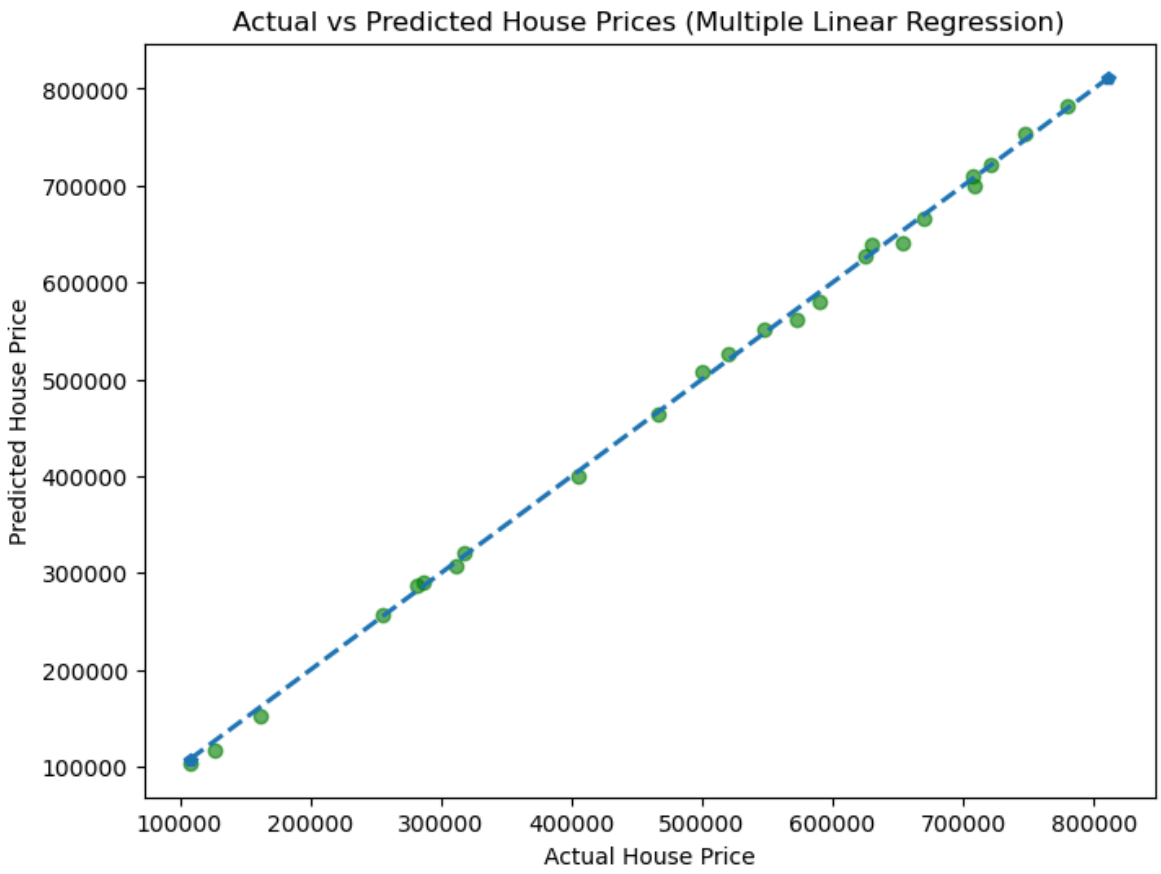
ASSIGNMENT NO 11 [Multiple Linear Regression]

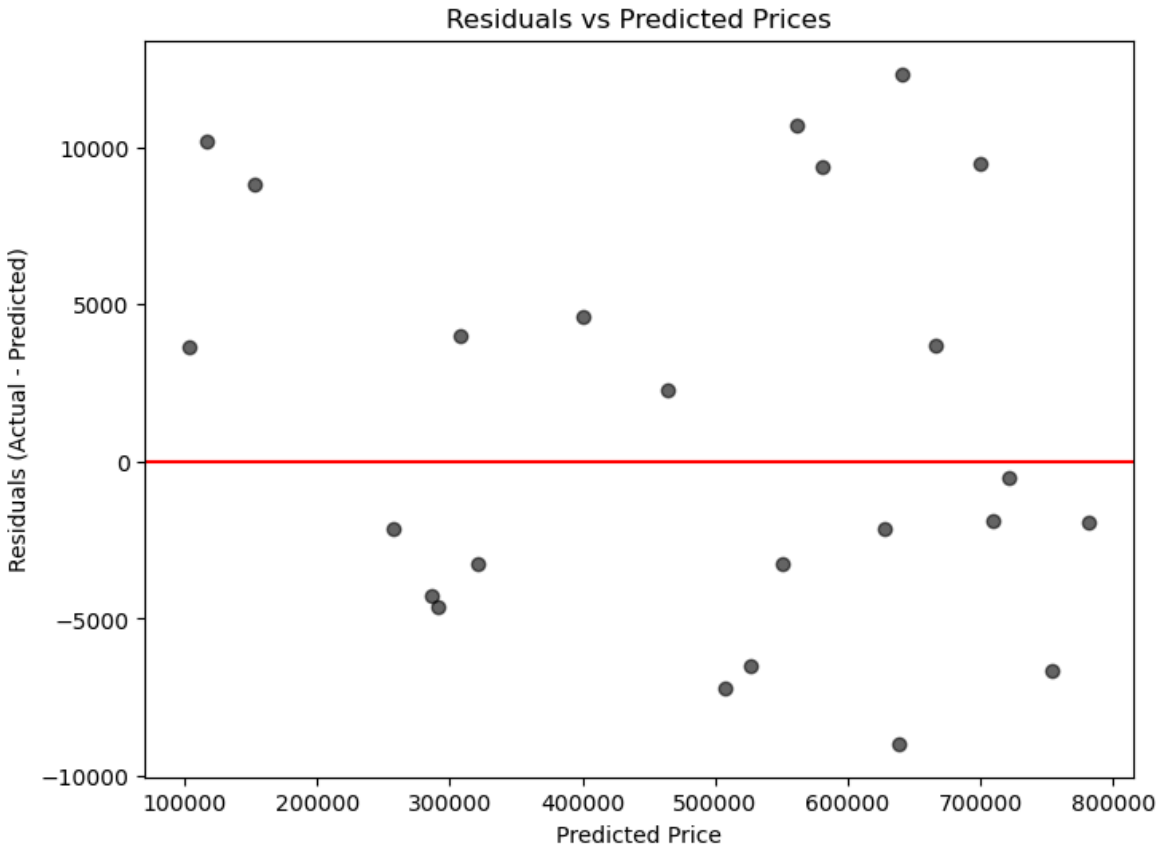
In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
# ----------------------------
# Step 1: Create Synthetic House Price Dataset
# ----------------------------
np.random.seed(42)
n = 120  # number of records

# Features
size = np.random.randint(500, 4000, n)        # Size in sqft
bedrooms = np.random.randint(1, 6, n)         # Number of bedrooms
bathrooms = np.random.randint(1, 4, n)        # Number of bathrooms
distance = np.random.randint(1, 30, n)        # Distance to city center (km)

# Target variable (House Price) with noise
price = (
    3000*bedrooms +
    5000*bathrooms +
    200*size -
    1500*distance +
    np.random.randint(-10000, 10000, n)  # noise
)

# Create DataFrame
house_data = pd.DataFrame({
    "Size_sqft": size,
    "Bedrooms": bedrooms,
    "Bathrooms": bathrooms,
    "Distance_km": distance,
    "Price": price
})

# Save dataset for future use
house_data.to_csv("house_prices.csv", index=False)
print("Dataset saved as house_prices.csv")
print(house_data.head())

# ----------------------------
# Step 2: Prepare Features & Target
# ----------------------------
X = house_data[["Size_sqft", "Bedrooms", "Bathrooms", "Distance_km"]]  # Indepen
y = house_data["Price"]                                                 # Depende

# Split data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# Step 3: Train Model
# ----------------------------
model = LinearRegression()
model.fit(X_train, y_train)

# ----------------------------
# Step 4: Predictions & Evaluation
# ----------------------------
y_pred = model.predict(X_test)

# Model performance
mse = mean_squared_error(y_test, y_pred)
```

```python
r2 = r2_score(y_test, y_pred)

print("\n 🔹 Model Coefficients (Weights):", model.coef_)
print(" 🔹 Intercept (β₀):", model.intercept_)
print(" 🔹 Mean Squared Error (MSE):", mse)
print(" 🔹 R² Score:", r2)

# Regression Equation
print("\nEquation:")
print(f"Price = {model.intercept_:.2f} "
      f"+ ({model.coef_[0]:.2f} * Size_sqft) "
      f"+ ({model.coef_[1]:.2f} * Bedrooms) "
      f"+ ({model.coef_[2]:.2f} * Bathrooms) "
      f"+ ({model.coef_[3]:.2f} * Distance_km)")

# ----------------------------
# Step 5: Visualization
# ----------------------------
# Actual vs Predicted plot
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, color="g", alpha=0.6)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'p--', lw=2)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted House Prices (Multiple Linear Regression)")
plt.show()

# Residual plot
residuals = y_test - y_pred
plt.figure(figsize=(8,6))
plt.scatter(y_pred, residuals, color="k", alpha=0.6)
plt.axhline(0, color="red", linestyle="-")
plt.xlabel("Predicted Price")
plt.ylabel("Residuals (Actual - Predicted)")
plt.title("Residuals vs Predicted Prices")
plt.show()
```

```
Dataset saved as house_prices.csv
   Size_sqft  Bedrooms  Bathrooms  Distance_km   Price
0       3674         3          3           24  720930
1       1360         3          3           18  278531
2       1794         1          2           15  340979
3       1630         3          3           22  310506
4       1595         3          3           23  311784

 🔹 Model Coefficients (Weights): [  199.63493784  2905.34423713  4464.78588106 -
1400.54505448]
 🔹 Intercept (β₀): -513.5268167564063
 🔹 Mean Squared Error (MSE): 41255013.98912258
 🔹 R² Score: 0.9990211626451463

Equation:
Price = -513.53 + (199.63 * Size_sqft) + (2905.34 * Bedrooms) + (4464.79 * Bathro
oms) + (-1400.55 * Distance_km)
```



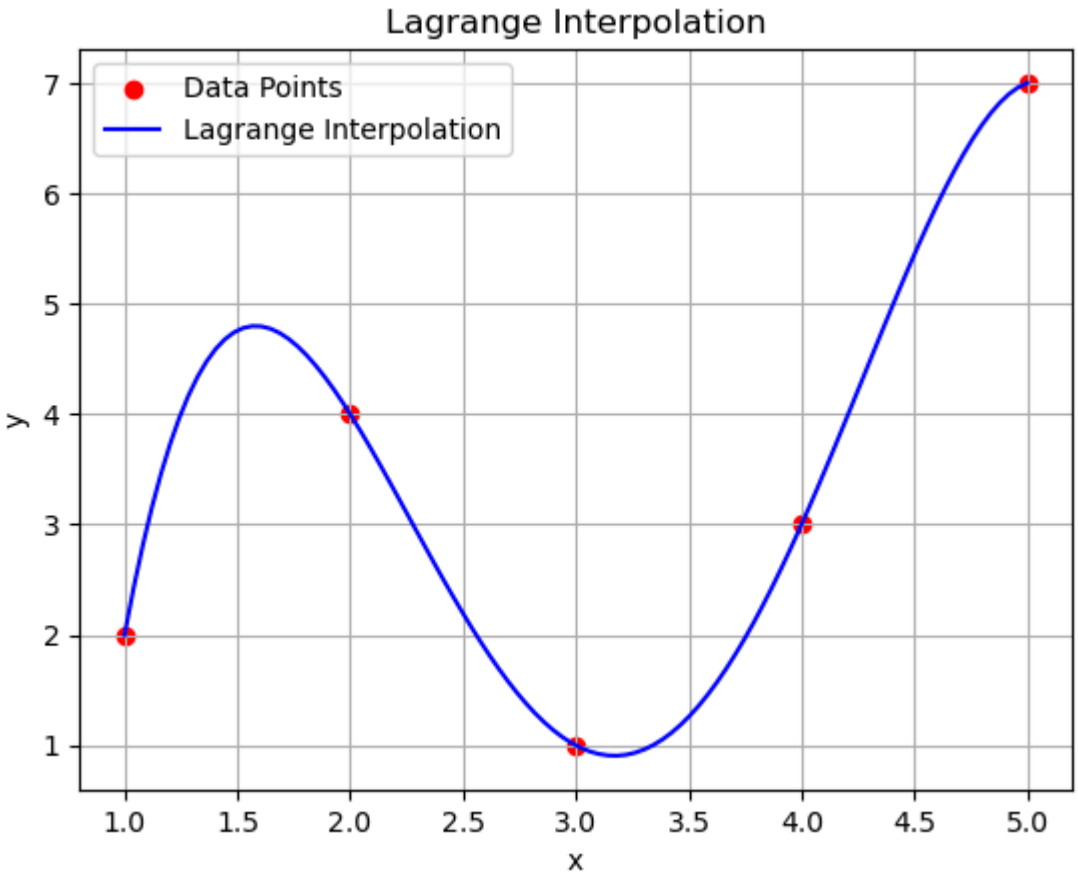


Explination : Multiple Linear Regression is a method used to predict one value using two or more other related values. It shows how different factors together affect the result.

ASSIGNMENT NO 12 [Lagrange Interpolation]

In [3]:
```
# Lagrange Interpolation in Python
import numpy as np
import matplotlib.pyplot as plt

# Sample data points (x, y)
x = np.array([1, 2, 3, 4, 5])       # x-coordinates
y = np.array([2, 4, 1, 3, 7])       # y-coordinates

# Function to perform Lagrange interpolation
def lagrange_interpolation(x_points, y_points, x_val):
    """
    x_points: array of x data points
    y_points: array of y data points
    x_val: the x-value at which interpolation is to be calculated
    Returns: interpolated y-value at x_val
    """
    n = len(x_points)
    result = 0.0

    # Iterate through each data point
    for i in range(n):
        # Compute the Lagrange basis polynomial L_i(x)
        term = y_points[i]
        for j in range(n):
            if i != j:
                term *= (x_val - x_points[j]) / (x_points[i] - x_points[j])
        result += term  # Add current term to result
    return result

# Interpolate at a new value
x_new = 2.5
y_new = lagrange_interpolation(x, y, x_new)
print(f"Interpolated value at x = {x_new} is y = {y_new}")

# Plotting the data points and interpolation curve
x_range = np.linspace(min(x), max(x), 100)  # Smooth curve
y_range = [lagrange_interpolation(x, y, xi) for xi in x_range]

plt.scatter(x, y, color='red', label='Data Points')  # Original data points
plt.plot(x_range, y_range, color='blue', label='Lagrange Interpolation')  # Inte
plt.xlabel('x')
plt.ylabel('y')
plt.title("Lagrange Interpolation")
plt.legend()
plt.grid(True)
plt.show()
```

```
Interpolated value at x = 2.5 is y = 2.1953125
```



This code performs Lagrange Interpolation to estimate a value between given data points. It calculates the interpolated value at a chosen x using a custom function and then plots both the original points and the smooth interpolation curve. The graph helps visualize how the interpolation fits the data.

ASSIGNMENT NO - 13 [Newton's Interpolation Method]

In [4]:
```
# Newton's Interpolation Method in Python

# Function to calculate the divided difference table
def divided_diff(x, y):
    """
    Calculates the divided difference table for Newton's Interpolation.
    x: list of x values
    y: list of y values (f(x))
    returns: a 2D list representing the divided difference table
    """
    n = len(y)
    # Create a 2D list initialized with zeros
    table = [[0 for _ in range(n)] for __ in range(n)]

    # First column is y values
    for i in range(n):
        table[i][0] = y[i]

    # Calculate divided differences
    for j in range(1, n):
        for i in range(n - j):
            table[i][j] = (table[i+1][j-1] - table[i][j-1]) / (x[i+j] - x[i])

    return table
```

```python
# Function to apply Newton's Interpolation formula
def newton_interpolation(x, y, value):
    """
    Computes the interpolated value at 'value' using Newton's Interpolation.
    x: list of x values
    y: list of y values
    value: the point at which interpolation is required
    returns: interpolated value at x = value
    """
    n = len(x)
    # Get divided difference table
    table = divided_diff(x, y)

    # Start with first y value
    result = table[0][0]

    # Compute interpolation
    product_term = 1.0
    for i in range(1, n):
        product_term *= (value - x[i-1])
        result += table[0][i] * product_term

    return result

# Example usage:
x_points = [1, 2, 3, 4]           # x values
y_points = [1, 4, 9, 16]          # f(x) = x^2

# Interpolate at x = 2.5
value_to_interpolate = 2.5
interpolated_value = newton_interpolation(x_points, y_points, value_to_interpola
print(f"Interpolated value at x = {value_to_interpolate} is {interpolated_value}
```

```
Interpolated value at x = 2.5 is 6.25
```

This code uses Newton's Interpolation Method to find a value between given data points. It first builds a divided difference table and then uses it to calculate the interpolated value at any desired point. Finally, it prints the estimated result showing how smoothly the data fits the interpolation curve.